ASM_code_review

ASM-VLN 代码评审文档

> **分支**: `zktian3`
> **定位**: 基于仿真的语义地图构建与视觉语言导航系统
> **代码规模**: ~16,400 行 Python，42 个源文件
> **生成日期**: 2026-04-23

一、项目概述

ASM-VLN（Active Semantic Mapping for Vision-Language Navigation）是一个室内仿真环境下的语义地图构建与自然语言导航系统。系统基于 Habitat-Sim 仿真器，通过 RGB-D 传感器采集环境数据，结合 GroundingDINO（开放词汇检测）、MobileSAM（实例分割）和 VLM（视觉语言模型）进行语义理解，实现：

1. **实时障碍物地图构建** — 深度点云投影生成俯视障碍物栅格
2. **语义物体检测与标注** — 物体检测 + 分割 → 语义值图
3. **房间区域分割** — 基于距离变换 + 分水岭的区域划分
4. **房间类型分类** — 物体先验 + VLM 分类融合推理
5. **VLM 驱动的自主导航** — 大模型决策下一步动作（转向/前进/停止）

二、目录结构

```
asm-vln/
├── nav_with_asm.py              # [370行] 主入口，组装模块并启动导航
├── make_asm_with_navmesh.py     # [1113行] 离线建图（navmesh扫描点）
├── make_asm_and_room_classfy.py # [1795行] 离线建图 + 房间分类完整流水线
│
├── navigation/                  # 导航核心模块
│   ├── simulator.py             # [305行] Habitat-Sim 封装
│   ├── slam_map.py              # [541行] SLAM地图管理器（协调各子地图）
│   ├── room_semantics.py        # [952行] 房间语义推理与融合
│   ├── nav_runner.py            # [491行] 在线导航主循环
│   ├── nav_mesh_runner.py       # [324行] NavMesh离线扫描循环
│   └── video_recorder.py        # [94行]  视频录制
```

```
13  │
14  ├── application/                  # 智能体
15  │   └── nav_agent.py              # [244行] VLM驱动的导航决策Agent
16  │
17  ├── asm_mapping/                  # 语义建图核心
18  │   ├── value_map.py              # [729行] 语义值图（物体存储与可视化）
19  │   ├── map_partation.py          # [1028行] 地图分割算法引擎
20  │   ├── realtime_partitioner.py   # [152行] 实时分割封装
21  │   ├── room_classifier.py        # [631行] VLM房间类型分类器
22  │   ├── object_detect.py          # [107行] 物体检测入口（GroundingDINO + SAM）
23  │   ├── object_detect_wedetect.py# [111行] 物体检测入口（WeDetect + SAM，远程服
    务）
24  │   ├── detect2grub/              # 检测服务通信层
25  │   │   ├── server_wrapper.py     # [164行] HTTP客户端与模型托管
26  │   │   ├── grounding_dino.py     # [196行] GroundingDINO RPC客户端
27  │   │   ├── sam.py                #         MobileSAM RPC客户端
28  │   │   └── detections.py         # [256行] 检测结果数据结构
29  │   └── mapping/                  # 底层地图算法
30  │       ├── base_map.py           # [111行] 地图基类（坐标变换）
31  │       ├── obstacle_map.py       # [215行] 障碍物地图
32  │       ├── geometry_utils.py     # [271行] 几何工具函数
33  │       ├── img_utils.py          # [390行] 图像处理工具
34  │       └── traj_visualizer.py    # [167行] 轨迹可视化
35  │
36  ├── envs/                         # 环境补丁
37  │   └── nav_sim_patch.py          # [23行]  Monkey-patch注入NavAgent
38  │
39  ├── util/                         # 工具库
40  │   ├── config_loader.py          # [20行]  YAML配置单例
41  │   ├── vlm.py                    # [96行]  VLM通信客户端
42  │   ├── logger.py                 #         日志模块
43  │   ├── image.py                  #         图像编码工具
44  │   └── crop_map.py               # [183行] 地图裁剪工具
45  │
46  ├── tools/                        # 工具脚本
47  │   ├── scan_object_frequency.py # [731行] 跨场景物体频率统计
48  │   ├── nav_with_asm_multi.py     # [1004行] 批量多路径导航评估
49  │   ├── navmesh_point_selector.py# [593行] NavMesh采样点选择器
50  │   ├── build_obstacle_maps_from_navmesh.py  # [475行] 批量障碍物地图构建
51  │   ├── check_asm.py              # [299行] 语义地图检查
52  │   ├── fix_asm.py                # [322行] 语义地图修复
53  │   ├── verify_objects_interactive.py # [338行] 交互式物体验证
54  │   ├── extract_semantic_data.py #         语义数据提取
55  │   ├── visualize_semantic.py     # [146行] 语义可视化
56  │   └── main.py                   # [97行]  工具入口
57  │
58  ├── scripts/                      # Shell脚本
```

```
59  │    ├── run_batch_navmesh.sh     # 批量navmesh建图
60  │    ├── run_build_obstacle_maps.sh # 批量障碍物地图
61  │    ├── run_scan_freq.sh         # 物体频率扫描
62  │    └── vllm_run_*.sh            # 不同VLM模型启动脚本
63  │
64  ├── config/                       # 配置文件
65  │    ├── GroundingDINO_SwinT_OGC.py # GroundingDINO模型配置
66  │    └── training_args.yaml       # 训练参数
67  │
68  ├── data/
69  │    └── class.txt                # 目标检测类别列表（自动生成）
70  │
71  ├── navmesh/                      # NavMesh扫描点配置（JSON）
72  ├── config.yaml                   # 全局配置文件
73  ├── run_port.sh                   # 检测服务启动脚本
74  └── vllm_run.sh                   # VLM推理服务启动脚本
```

三、系统架构

3.1 整体数据流

```
1   ┌──────────────────────────────────────────────────────────────────┐
2   │                    用户输入："去卧室"                              │
3   └──────────────────────────┬──────────────────────────────────────┘
4                              ▼
5             ┌──────────────────────────┐
6             │   nav_with_asm.py        │  主入口：参数解析、模块组装
7             │   (模式选择)              │  → auto / manual / navmesh
8             └──────────────┬───────────┘
9                              ▼
10    ┌──────────────────────────────────────────────────────────┐
11    │              NavRunner / NavMeshRunner                    │
12    │              (导航主循环 - 每步执行)                        │
13    │                                                           │
14    │  ┌──────────┐    ┌──────────────┐    ┌────────────────┐  │
15    │  │Habitat   │──▶│ SlamMap      │──▶│ RoomSemantics  │  │
16    │  │Simulator │    │ Manager      │    │ Analyzer       │  │
17    │  │(RGB+D)   │    │              │    │                │  │
18    │  └──────────┘    └──────────────┘    └────────────────┘  │
19    │      │              │                  │          │
20    │      ▼              ▼                  ▼          │
21    │  ┌──────────┐    ┌──────────────┐    ┌────────────────┐   │
22    │  │NavAgent  │    │Obstacle      │    │Room            │   │
```

```
    │  │(VLM    │   │Map +     │   │Classifier    │   │
    │  │决策)   │   │ValueMap  │   │(VLM分类)     │   │
    │  └────────┘   └──────────┘   └──────────────┘   │
    └──────────────────────────────────────────────────┘
```

3.2 核心模块关系

```
nav_with_asm.py
  │
  ├── HabitatSimulator          传感器配置、位姿查询、动作执行
  │     └── NavAgent            VLM决策Agent（通过monkey-patch注入）
  │           └── VLM           大模型通信客户端
  │
  ├── SlamMapManager            SLAM地图总协调
  │     ├── ObstacleMap         深度→障碍物栅格（BaseMap子类）
  │     ├── ValueMap            语义物体地图（存储、去重、可视化）
  │     ├── RealObjectDetector  物体检测（两套后端二选一）
  │     │     ├── [后端A] GroundingDINOClient ── HTTP localhost:12181
  │     │     ├── [后端B] WeDetectClient      ── HTTP 远程:12181
  │     │     └── MobileSAMClient             ── HTTP :12183（两套共用）
  │     └── RealtimePartitioner 实时区域分割
  │           └── MapPartitioner  分水岭分割算法引擎
  │
  ├── RoomSemanticsAnalyzer     房间语义推理
  │     └── RoomClassifier      VLM多方位图像房间分类
  │
  ├── NavRunner                 在线导航循环（auto/manual）
  ├── NavMeshRunner             NavMesh离线扫描循环
  └── VideoRecorder             视频录制
```

3.3 坐标系统

系统涉及四套坐标系，`SlamMapManager` 提供双向转换：

坐标系	原点	说明
Habitat世界坐标 (x, y, z)	场景原点	y轴朝上，仿真器原生坐标
相机坐标 (x_c, y_c, z_c)	相机中心	x右、y下、z前
全局地图像素 (px, py)	地图中心	`pixels_per_meter=30`，双轴翻转
裁剪分区像素 (px', py')	裁剪区域左上角	分区可视化专用

关键转换函数：

- `SlamMapManager.habitat_to_full_map_pixel()` /
- `SlamMapManager.habitat_to_partition_pixel()` / `partition_pixel_to_habitat()`
- `ValueMap.world_to_map_coords()` / `compute_habitat_position()`

四、核心模块详解

4.1 导航决策 — `NavAgent`

项目	说明
位置	`application/nav_agent.py` (244行)
继承	`habitat_sim.agent.Agent`
注入方式	`envs/nav_sim_patch.py` monkey-patch `Simulator._config_agents`

状态机： `Agent_Mode` — NORMAL → RECOVER → STOP / ERROR

决策流程：

1. 将当前 RGB、历史帧、ASM 鸟瞰图编码为 base64
2. 发送至 VLM（Qwen3-VL）获取中文指令
3. 解析响应为仿真器动作（`turn_left` / `turn_right` / `move_forward` / `stop`）
4. 震荡/卡住检测 → 自动进入 RECOVER 模式

后台判断： `judge_real_arrive()` 线程定期调用 VLM 判断是否到达目标

4.2 ASM 构建完整流水线

ASM的构建贯穿整个导航过程，由 `SlamMapManager` 统一调度，分为五个阶段：

```
阶段1：障碍物地图        阶段2：物体检测          阶段3：语义值图          阶段4：区域分割
        阶段5：房间语义

 Depth ──────────▶ RGB ──────────▶ 检测结果 ──────────▶ 障碍物地图 ──────▶ 分区结果 ──────▶
```

```
 ┌──────────┐      ┌──────────────┐      ┌──────────────┐      ┌──────────────┐      ┌──────────────┐
 │ObstacleMap│      │GroundingDINO │      │  ValueMap    │      │MapPartitioner│      │RoomSemantic│
 │  .update  │      │  + SAM       │      │ .add_object  │      │ .partition   │      │ sAnalyzer  │
 │  _map()   │      │  → bbox+mask │      │ → 坐标+去重  │      │ → 区域 mask  │      │ .update()  │
 └──────────┘      └──────────────┘      └──────────────┘      └──────────────┘      └──────────────┘

                          │
                          ▼
                   带房间类型标签的
                   彩色语义地图
```

阶段 1：障碍物地图构建 — `ObstacleMap`

项目	说明
位置	`asm_mapping/mapping/obstacle_map.py` (215 行)，继承 `BaseMap`
触发	每步调用 `SlamMapManager.update()`

处理步骤：

```
Depth图 (归一化 [0,1])
  │
  ├─ fill_small_holes() ──────── 填充深度小空洞
  │
  ├─ 反归一化 → 真实深度(m)
  │
  ├─ get_point_cloud(depth, mask, fx, fy) ── 像素 → 相机坐标系3D点云
  │
  ├─ transform_points(tf_matrix, points) ── 相机坐标 → episodic坐标（世界坐标）
  │
  ├─ filter_points_by_height(min=0.1m, max=2.0m) ── 高度过滤，仅保留障碍物范围
  │
  ├─ _xy_to_px() ─── 世界XY → 地图像素坐标
  │
```

```
    ├─ 写入 self._map[y, x] = 1 ─── 标记障碍物像素
    │
    ├─ cv2.dilate(obstacle, kernel) ── 按agent半径膨胀障碍区域
    │
    ├─ self._navigable_map = 1 - dilated ── 可通行区域 = 非障碍区域
    │
    ├─ reveal_fog_of_war() ─── 基于agent位姿 + FOV计算当前可见探索区域-
    │
    └─ detect_frontier_waypoints() ── 计算已探索区域边界的前沿点
```

输出：`self._map`（障碍物栅格）、`self.explored_area`（已探索区域）、`self._navigable_map`（可通行区域）、`self.frontiers`（前沿点）

关键参数：

- 地图尺寸 `1000×1000` 像素，`pixels_per_meter=30`，覆盖约 33m × 33m
- 障碍物高度过滤范围 `[0.1m, 2.0m]`（由 `config.yaml` 的 `min_height` / `max_height` 决定）
- `SlamMapManager.update()` 中 `obstacle_yaw = camera_yaw + π/2`（坐标系对齐）

阶段 2：物体检测 — `RealObjectDetector`

项目	说明
触发条件	`step_count % detection_frequency == 0`（默认每5步检测一次）
入口	`SlamMapManager.detect_and_update_objects()`

检测流程：

```
当前RGB帧
  │
  ├─ GroundingDINO / WeDetect ──▶ ObjectDetections
  │    caption = "chair . table . bed . sofa ..."   (来自 data/class.txt)
  │    → boxes (归一化xyxy), logits (置信度), phrases (类别名)
  │
  ├─ filter_by_class(target_classes) ── 过滤非目标类别
  │
  ├─ filter_by_conf(0.4) ── 置信度阈值过滤
  │
  └─ 逐物体循环:
```

```
        ├─ MobileSAM.segment(rgb, bbox) ──▶ 像素级分割mask
        │
        └─ ValueMap.add_object(object_info, camera_pos, yaw, robot_xy)
            传入: name, mask, bbox, confidence, depth, camera_params,
                  habitat_agent_position, camera_height
```

类别列表来源：`data/class.txt`（由 `tools/scan_object_frequency.py` 跨场景统计生成），一行一个英文类别名，拼接为 `"chair . table . bed ."` 格式传入检测器。

阶段 3：语义值图写入 — `ValueMap.add_object()`

项目	说明
位置	`asm_mapping/value_map.py` (729行)
核心数据	`detected_objects` 列表 + `object_regions` 2D 数组

单个物体写入流程（`add_object()` 方法，核心逻辑）：

```
输入: object_info (name, mask, bbox, depth, confidence, camera_params, habitat_agent_pos)
  │
  ├─ ① get_object_center_depth(mask, depth, bbox)
  │     bbox区域内取mask有效深度的中位数 (Q25/Q50/Q75)
  │     IQR > 2.0m → 拒绝 (深度跳变太大，不可靠)
  │     深度范围 [0.3m, 6.0m], 有效点 < 5 → 拒绝
  │     → center_x, center_y, center_depth
  │
  ├─ ② extract_object_region_points(mask, depth, camera_params, ...)
  │     遍历mask非零点 (每3像素采样一次以提速)
  │     每个点: pixel → camera坐标(pixel_to_world_ray)
  │           → 世界坐标(cos/sin旋转) → 地图像素坐标(world_to_map_coords)
  │     → region_points: [(map_x, map_y, world_x, world_z), ...]
  │
  ├─ ③ pixel_to_world_ray() → camera_point (相机坐标系3D)
  │     cos/sin旋转 → 地图坐标系 (world_x, world_z) → (map_x, map_y)
  │
  ├─ ④ compute_habitat_position(camera_point, yaw, habitat_agent_pos, camera_height)
  │     相机坐标 → 旋转到世界坐标 → 加上agent位置 → habitat坐标
  │     Y轴: habitat_agent_y + camera_height - camera_y  (Y-down → Y-up)
  │     Z轴: 取反 (坐标系惯例)
  │     最终钳位到 navmesh bounds 内 (防止深度误差导致坐标越界)
```

```
    │
    ├─ ⑤ _should_skip_object(name, map_x, map_y, min_distance=60px)
    │      遍历已有物体，同名/同义词 + 距离<60px → 跳过（语义去重）
    │      同义词组: {chair,seat,stool}, {sofa,couch}, {tv,television,monitor},
...
    │
    ├─ ⑥ merge_overlapping_regions(region_points, name, color, confidence)
    │      检查新点在 object_regions 中与现有区域的重叠度
    │      重叠率 > 30% 且同名 → 合并到现有物体（更新置信度）
    │      否则 → 分配新 object_id
    │
    ├─ ⑦ 写入 object_regions[y, x] = object_id  （区域涂色）
    │
    └─ ⑧ 添加/更新 detected_objects 条目
         新物体: 完整记录 (object_id, name, position, habitat_position, ...)
         已有物体: 置信度加权融合 habitat_position，保留最高置信度的 position/depth
```

物体数据结构（`detected_objects` 条目）：

```python
{
    "object_id": 1,                     # 唯一ID，用于 object_regions 索引
    "name": "sofa",                     # 检测类别名
    "position": (512, 487),             # 地图像素坐标（用于可视化标签）
    "habitat_position": [1.2, 0.4, -3.1],  # Habitat世界坐标 [x, y, z]
    "robot_position": [0.8, 0.0, -2.9],    # 检测时agent的Habitat坐标
    "size_2d": {"width": 0.85, "height": 0.62},  # 物体2D尺寸估算(m)
    "area": "living_room",              # 所属房间类型（阶段5填入）
    "color": (255, 255, 0),             # 可视化颜色
    "confidence": 0.78,                 # 检测置信度
    "depth": 2.35,                      # 中心点深度(m)
    "scan_id": 3,                       # NavMesh扫描点ID（navmesh模式）
    "angle_index": 2,                   # 扫描角度索引（navmesh模式）
}
```

坐标转换链：

```
像素 (u,v)  ──pixel_to_world_ray──▶  相机坐标 (cx,cy,cz)
                                          │
                      cos/sin(yaw) 旋转 + robot_xy 平移
                                          │
                                          ▼
                                     地图世界坐标 (world_x, world_z)
                                          │
```

```
                    world_to_map_coords (× pixels_per_meter + center)
                                        │
                                        ▼
                                  地图像素坐标 (map_x, map_y)

像素 (u,v)  ──pixel_to_world_ray──▶  相机坐标 (cx,cy,cz)
                                        │
                    compute_habitat_position (旋转 + agent位置 + 高度补偿)
                                        │
                                        ▼
                                  Habitat坐标 (x, y, z)
```

阶段 4：区域分割 — `MapPartitioner`

项目	说明
位置	`asm_mapping/map_partation.py` (1028行)
封装	`RealtimePartitioner` → `SlamMapManager.run_partition()`
触发	导航过程中按 `partition_frequency` 定期触发

预处理（`SlamMapManager.prepare_partition_map()`）：

```
ObstacleMap.visualize()
  │
  ├─ cv2.rotate(ROTATE_90_CLOCKWISE) ── 旋转对齐
  │
  ├─ get_black_pixel_bounds() ── 检测有效区域边界
  │
  └─ crop_by_black_pixels() + flip_vertical ── 裁剪 + 翻转 → partition_map
```

分割算法（`MapPartitioner.partition()`，6步流程）：

```
Step 1: 二值化 + 形态学预处理
  │  threshold(binarization_threshold=250) → 自由空间/障碍物
  │  morphologyEx(CLOSE, wall_closing_size) → 闭合墙壁小缝隙
  │  morphologyEx(OPEN, cleanup_opening_size) → 清理噪声
  │
Step 2: 距离变换 → 种子点选择  (_select_seed_points)
  │  distance_transform_edt(free_space) → 每个自由像素到最近墙壁的距离
```

```
   │  GaussianBlur(sigma) → 平滑距离图
   │  峰值检测: 距离 > min_seed_clearance(0.35m) 的局部极大值
   │  过滤: 种子间距 > min_seed_distance(1.2m)
   │  走廊补充: 长条区域额外按 corridor_seed_spacing(1.8m) 间隔加点
   │  → seed_points: [(x, y), ...]
   │
Step 3: 分水岭初始分割  (_initial_segmentation)
   │  以种子点为标记，对距离图取负值作为地势图
   │  cv2.watershed() → 初始区域标签
   │
Step 4: 竞争式区域生长  (_competitive_grow_labels)
   │  从种子向外 BFS 扩展（8连通），用优先队列按代价排序
   │  窄通道惩罚: 通过窄通道的代价 × narrow_passage_penalty(2.0)
   │  → 填充所有未标记的自由空间像素
   │
Step 5: 区域过滤与合并
   │  _remove_tiny_regions(): 面积 < min_room_area(0.75m²) 的区域合并到相邻最大区域
   │  _filter_abnormal_regions(): 面积 > max_reasonable_room_area(30m²) 的异常区域重新分割
   │  _build_region_graph(): 构建区域邻接图（共享边界长度、比例）
   │  _merge_regions(): 评分驱动合并
   │     score = f(面积比, 边界共享率, 形状紧凑度, 走廊因子)
   │     score > merge_score_threshold(0.35) → 合并
   │
Step 6: 窄通道分裂  (_split_room_by_narrow_connections)
   │  面积 > split_candidate_area(6m²) 的区域
   │  检测距离图中核心区域（clearance > split_core_clearance=0.35m）
   │  若存在 ≥2 个间距 > 1.5m 的独立核心 → 沿窄通道切分
   │
   ▼
输出: List[RoomRegion]  每个区域包含 region_id, mask(bool矩阵), area, bbox, center
```

MapConfig 关键参数（物理单位）：

参数	默认值	含义
`pixels_per_meter`	30.0	分辨率
`min_room_area_m2`	0.75	最小房间面积，更小的被合并
`min_region_size_m2`	1.5	种子点生成的最小区域面积
`max_merge_area_m2`	20.0	合并后面积上限

`max_reasonable_room_area_m2`	30.0	超过此面积触发异常检测
`narrow_passage_width_m`	0.9	窄通道宽度阈值
`seed_prominence_m`	0.30	种子点距离变换突出度阈值

阶段 5：房间语义推理 — `RoomSemanticsAnalyzer`

项目	说明
位置	`navigation/room_semantics.py` (952行)
策略	物体先验打分 + VLM四方位图像分类 → 加权融合

完整推理流程：

```
Step 1: 物体分配 (_assign_objects_to_regions)
  │ 遍历 ValueMap.detected_objects，对每个物体:
  │   优先用 map_position → 备选 position → 最后 habitat_position 转换
  │   通过 full_map_pixel_to_partition_pixel() 转到分区坐标
  │   在 label_map 中查找所属 region_id（支持 search_radius 邻域搜索）
  │ → region_objects: {region_id: [object1, object2, ...]}
  │
Step 2: 物体先验推理 (_infer_room_type_from_objects)
  │ 对每个区域，遍历其物体列表:
  │   查询 ROOM_OBJECT_PRIORS 表（11类房间 × 多关键词权重）
  │   匹配方式: 精确匹配/子串包含/token子集 → 权重 × 物体置信度
  │ 累计各房间类型得分 → 归一化为概率分布
  │ 最高分房间类型即为先验结果
  │
  │ ROOM_OBJECT_PRIORS 示例:
  │   "bedroom": {"bed": 6.0, "mattress": 5.0, "pillow": 2.0, ...}
  │   "kitchen": {"refrigerator": 5.0, "stove": 5.0, "microwave": 3.0, ...}
  │   "bathroom": {"toilet": 6.0, "bathtub": 5.0, ...}
  │
Step 3: VLM 房间分类 (可选，_classify_region_semantics)
  │ 条件: enable_room_classifier=True 且先验置信度 < strong_prior_threshold(0.8)
  │ 对目标区域:
  │   计算区域内切圆中心 → Habitat坐标
  │   teleport agent到该坐标 → capture_four_directions() 拍摄前/后/左/右4张RGB
  │   RoomClassifier.classify_room(images) → {"room_type": "bedroom", "confidence": 0.85}
  │
```

```
    │  RoomClassifier 内部:
    │    4张图 → base64编码 → 组装prompt ("请根据图片判断房间类型")
    │    调用 VLM API (本地vLLM / SiliconFlow / DashScope, 三种后端)
    │    解析JSON响应 → room_type + confidence
    │    响应解析失败 → fallback_parse 正则匹配
    │
Step 4: 结果融合  (_fuse_room_predictions)
    │  策略:
    │    若先验置信度 ≥ strong_prior_threshold → 直接采用先验 (不调VLM)
    │    若 VLM 与先验一致 → 取较高置信度
    │    若不一致 → 加权融合, 权重可配置
    │
Step 5: 相邻同类合并  (_merge_adjacent_semantic_regions)
    │  遍历区域邻接图:
    │    相邻且同类型 → 合并mask + 重新计算属性
    │    合并后重新检查新的相邻关系 → 迭代直到无可合并
    │  → _refine_merged_regions() 处理合并后的边界清理
    │
Step 6: 可视化输出  (_create_visualization)
    │  每个区域填充 ROOM_COLOR_MAP 对应颜色
    │  叠加中文房间类型标签 (内切圆中心位置)
    │  叠加物体检测标签
    │  → 保存到 partitions/room_semantics/
    │
Step 7: 回写物体房间归属
    │  ValueMap.update_object_areas({object_id: room_type})
    │  → 每个物体的 "area" 字段被填入所属房间类型
    │  → 最终通过 export_semantic_data() 导出到 final_semantic_data.json
```

15类房间类型:

英文标识	中文	英文标识	中文
`living_room`	客厅	`balcony`	阳台
`bedroom`	卧室	`storage`	储藏室
`kitchen`	厨房	`laundry`	洗衣房
`bathroom`	浴室	`garage`	车库
`dining_room`	餐厅	`office`	办公室
`study`	书房	`gym`	健身房
`hallway`	走廊	`library`	图书馆

`unknown`	未知		

阶段调度：`SlamMapManager.update()` 每步执行

```
def update(self, step_count):
    # 1. [每步] 获取 RGB-D 观测 + agent位姿
    obs = simulator.get_observations()
    depth, camera_position, camera_yaw, robot_xy = ...

    # 2. [每步] 障碍物地图更新
    tf = xyz_yaw_to_tf_matrix(camera_position, yaw + π/2)
    obstacle_map.update_map(depth_normalized, tf, ...)
    obstacle_map.update_agent_traj(robot_xy, yaw)

    # 3. [每 detection_frequency 步] 物体检测
    if step_count % detection_frequency == 0:
        detect_and_update_objects(step_count)

    # 4. [每5步] 存入观测池 (供真机路径房间分类使用)
    observation_pool.append({"rgb": rgb, "habitat_position": pos, ...})

    # 5. [每步] 合成可视化: 障碍物地图叠加语义标签
    obstacle_vis → rotate_90 → value_map.visualize(base_map) → reorient_rescale

    # 6. [每步] 推送到 map_update_queue (供 NavAgent 消费鸟瞰图)
    map_update_queue.put(map_pil)
```

分区与房间语义由 `NavRunner._run_partition_with_semantics()` 按 `partition_frequency` 单独调度：

```
# NavRunner 中:
if step_count % partition_frequency == 0:
    partition_result = slam_map.run_partition(step_count, prefix)
    if partition_result:
        room_sem.update(partition_result, partition_context, step_count)
```

4.6 物体检测服务 — 双后端架构

系统提供两套检测后端，共享相同的 `RealObjectDetector` 接口（`detect()` + `segment()`），由不同入口文件选择：

后端	入口文件	检测客户端	使用场景
GroundingDINO	`object_detect.py` (107行)	`GroundingDINOClient`	本地GPU服务，高精度
WeDetect	`object_detect_wedetect.py` (111行)	`WeDetectClient`	远程检测服务，无需本地GPU

两者的 `RealObjectDetector` 类名相同，通过 import 路径切换：

```
# 方式一: GroundingDINO
from asm_mapping.object_detect import RealObjectDetector

# 方式二: WeDetect
from asm_mapping.object_detect_wedetect import RealObjectDetector
```

检测流程：

```
RGB图像 ──▶ GroundingDINO (:12181)  ──▶ 检测框 + 类别
          或  WeDetect (远程:12181)
                                          |
检测框 ───▶ MobileSAM (:12183)     ──▶ 实例分割掩码
```

WeDetect 适配要点（`WeDetectClient`，位于 `detect2grub/grounding_dino.py:88-176`）：

- WeDetect `/infer` 接口每次只接受一个 query，因此对 caption 中的每个类别分别发请求后合并结果
- WeDetect 返回绝对像素坐标 xyxy，客户端归一化到 `[0,1]` 以保持与 GroundingDINO 输出格式一致
- 通过环境变量配置：`WEDETECT_HOST`（默认 `192.168.31.180`）、`WEDETECT_PORT`（默认 `12181`）、`WEDETECT_SCORE_THRE`（默认 `0.3`）
- 使用 `requests.Session` 并禁用系统代理（`trust_env=False`）避免 socks 代理问题

通信层 `detect2grub/server_wrapper.py` 提供：

- 图像 base64 编解码
- 文件锁防并发冲突
- 请求重试机制

检测结果数据结构 — 两套后端各自定义了 `ObjectDetections`：

- `detect2grub/detections.py` (256行)：GroundingDINO 后端使用，基于 torch tensor，支持 JSON 序列化
- `object_detect_wedetect.py` 内联定义：WeDetect 后端使用，基于 numpy array，结构更简化

五、三种运行模式

模式一：自动导航（Auto）

```
python nav_with_asm.py --scene <scene.glb> --prompt "去卧室" --mode auto
```

VLM 驱动的端到端导航：感知 → 建图 → 决策 → 执行 → 循环。

模式二：手动控制（Manual）

```
python nav_with_asm.py --scene <scene.glb> --mode manual
```

键盘 WASD 控制移动，系统自动建图和物体检测，用于调试。

模式三：NavMesh 离线建图

```
python nav_with_asm.py --scene <scene.glb> --mode navmesh --points navmesh/00054.json
```

按预定义扫描点遍历场景，多角度拍摄 → 物体检测 → 建图 → 分区 → 房间分类。

六、外部服务依赖

服务	端口	用途	启动方式
GroundingDINO	12181 (本地)	开放词汇物体检测	`run_port.sh` （默认注释）
WeDetect	12181 (远程)	开放词汇物体检测（远程替代方案）	远程部署，环境变量 `WEDETECT_HOST` 配置

MobileSAM	12183	实例分割	`run_port.sh`
VLM (Qwen3-VL)	9000 (本地) 或 SiliconFlow API	导航决策 + 房间分类	`vllm_run.sh` 或远程 API

七、离线工具链

工具	说明
`make_asm_with_navmesh.py`	离线NavMesh建图，输出障碍物地图 + 语义数据
`make_asm_and_room_classfy.py`	完整流水线：建图 → 分区 → 房间分类 → 语义地图
`tools/scan_object_frequency.py`	跨场景VLM物体频率统计 → 生成 `data/class.txt`
`tools/nav_with_asm_multi.py`	批量多路径导航评估
`tools/build_obstacle_maps_from_navmesh.py`	批量构建障碍物地图
`tools/check_asm.py` / `fix_asm.py`	语义地图质量检查与修复
`tools/verify_objects_interactive.py`	交互式物体检测验证
`tools/navmesh_point_selector.py`	NavMesh采样点选择

八、输出产物

每次导航/建图生成 `slam_nav_YYYYMMDD_HHMMSS/` 目录：

```
slam_nav_20260410_181147/
├── maps/                    # 障碍物地图 + 语义地图图像
├── partitions/              # 区域分割掩码 + JSON统计
├── navigation/              # 轨迹数据
├── final_semantic_data.json # 完整语义数据（物体坐标、类别、房间归属）
└── video.mp4                # 导航过程录像
```

`final_semantic_data.json` 结构：

```json
{
  "objects": [
    {
      "name": "sofa",
      "habitat_pos": [x, y, z],
      "confidence": 0.85,
      "room_type": "living_room"
    }
  ]
}
```

九、关键设计决策

9.1 Monkey-Patch 注入 NavAgent

`envs/nav_sim_patch.py` 通过替换 `habitat_sim.Simulator._config_agents` 方法，使所有 Simulator 实例自动创建 `NavAgent` 而非默认 Agent。这是一个全局副作用——`import envs.nav_sim_patch` 即生效。

评审关注点：隐式全局修改，阅读代码时不易发现；如果其他模块需要原始 Agent 行为会受影响。

9.2 检测服务通信

物体检测采用 Client-Server 架构，通过 HTTP + base64 图像编码通信：

- 文件锁（`lockfiles/`）防止并发请求冲突
- 请求超时重试机制
- 锁文件可能残留（>120s 视为过期需手动清理）

评审关注点：文件锁可靠性、锁文件泄漏风险。

9.3 VLM 多后端支持

`RoomClassifier` 支持三种 VLM 调用方式：

- 本地 OpenAI 兼容 API（vLLM）
- OpenAI 格式远程 API（SiliconFlow）
- DashScope API（阿里云）

通过 `config.yaml` 中的 `vlm_base_url` 切换。

十、配置说明

核心配置文件 `config.yaml`：

配置项	说明
`vlm_base_url` / `vlm_model`	VLM 服务地址与模型名
`class_path`	检测类别列表路径（`data/class.txt`）
`habitat.camera_height`	相机高度，影响物体坐标计算
`habitat.hfov`	水平视场角
`habitat.turn_angle`	转向角度（度）
`habitat.forward_step_size`	前进步长（米）
`habitat.min_depth` / `max_depth`	深度有效范围

环境变量（无头渲染必须）：

```
export PYOPENGL_PLATFORM=egl
export MAGNUM_LOG=quiet
export HABITAT_SIM_LOG=quiet
```

十一、代码统计

模块	文件数	总行数	最大文件
`navigation/`	6	2,707	`room_semantics.py` (952)
`asm_mapping/`	15	4,718	`map_partation.py` (1028)
`application/`	1	244	`nav_agent.py` (244)
`util/`	5	~400	`crop_map.py` (183)
`tools/`	10	~4,100	`nav_with_asm_multi.py` (1004)

`envs/`	2	127	`__init__.py` (104)
根目录脚本	3	3,278	`make_asm_and_room_classfy.py` (1795)
合计	**42**	**~16,400**	